UNIVERSIDADE DO OESTE DE SANTA CATARINA
ÁREA DE CIÊNCIAS EXATAS E TECNOLÓGICAS

Curso - Engenharia de Computação
12 de Novembro de 2026 Componente - Inteligência Artificial e Sistemas Inteligentes

---

SPRINT 5

ANÁLISE DOS EXPERIMENTOS E RESULTADOS

Aluna: Ana Clara Viganó P. de Oliveira - 412887 **Aluna:** Jasmine Masson Alves - 427466

---

1. Introdução

Documentação técnica referente ao treinamento, avaliação e experimentação de um modelo de linguagem baseado na arquitetura GPT-2 implementado. O objetivo principal foi compreender a dinâmica de treinamento, o comportamento da função de perda (*Cross-Entropy Loss*) e as estratégias de decodificação, além de analisar quantitativa e qualitativamente o impacto de diferentes hiperparâmetros na convergência do modelo.

2. Avaliação e Implementação do Treinamento

Para quantificar o erro do modelo em prever o próximo token, utilizou-se a *Cross-Entropy Loss*. Essa métrica avalia a diferença entre a distribuição de probabilidade gerada pelo modelo (logits) e o token alvo real. Adicionalmente, implementou-se a métrica de *Perplexity* ($P = e^{loss}$), que traduz a incerteza do modelo de forma mais intuitiva, ou seja, uma perplexidade alta indica que o modelo está distribuindo sua "certeza"de forma quase aleatória entre diversas palavras do vocabulário.

O fluxo do treinamento foi construído iterando sobre lotes (*batches*) de um pequeno conjunto de dados (*the-verdict.txt*), realizando a propagação direta (*forward pass*), o cálculo do erro, a retropropagação (*backward pass*) e a atualização dos pesos utilizando o otimizador AdamW.

3. Experimentos de Treinamento

Para investigar a convergência e a qualidade da geração de texto, foram realizados três experimentos distintos, alterando a Taxa de Aprendizado (Learning Rate) e as técnicas de regularização.

Experimento: Taxa de Aprendizado Alta (LR = 0.01)

Neste experimento, teve-se como objetivo observar o impacto de passos de atualização muito grandes. O modelo foi treinado por 5 épocas.

- **Análise Quantitativa:** O treinamento demonstrou severa instabilidade. A *loss* de treinamento iniciou em 9.330, "explodiu"para 11.715 no passo 5, e encerrou em 6.391. A *loss* de validação seguiu a mesma instabilidade, chegando a 12.138 e terminando em 8.468.
- **Análise Qualitativa:** Devido às enormes variações dos pesos, o modelo perdeu a capacidade de formar estruturas semânticas, limitando-se a gerar sequências infinitas de vírgulas no final do treinamento (*"Every effort moves you,,,,,,,,..."*).

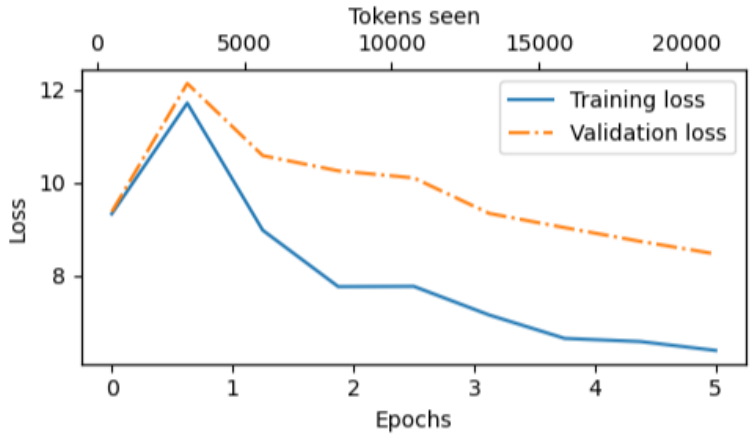


Figura 1 - Curvas de Loss para LR=0.01.

Experimento: Taxa de Aprendizado Baixa (LR = 1e-5)

Nesse experimento, a taxa de aprendizado foi reduzida drasticamente para observar o fenômeno de *underfitting* ao longo de 10 epochs.

- **Análise Quantitativa:** Observou-se uma convergência extremamente lenta e ineficiente. A *loss* de treinamento reduziu de 10.935 para apenas 8.561 na época 10. A *loss* de validação caiu de forma igualmente lenta (10.939 para 8.847).
- **Análise Qualitativa:** Com a *loss* ainda muito alta, o modelo sequer conseguiu aprender palavras completas ou gramática básica. O texto gerado estacionou em padrões de pontuação e artigos repetitivos (*"Every effort moves you,,,,,,, the,,,,,, the..."*).

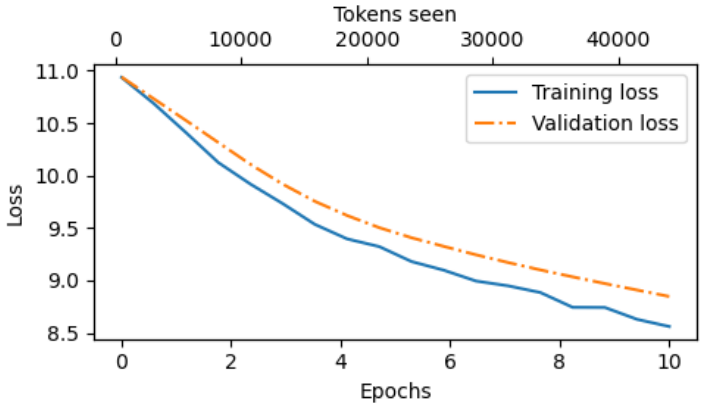


Figura 2 - Curvas de Loss para Taxa de Aprendizado Baixa (LR=1e-5).

Experimento: Regularização (*Dropout* 0.3, *Weight Decay* 0.5)

O objetivo deste experimento foi combater o forte *overfitting* percebido no código original (sem pesos do GPT-2), utilizando uma taxa de aprendizado equilibrada (0.0004), mas com forte penalização aos pesos (*weight decay*) e maior desligamento aleatório de neurônios (*dropout*).

- **Análise Quantitativa:** O modelo obteve excelente declínio na *loss* de treinamento, encerrando a época 10 em 2.154. Contudo, a *loss* de validação estabilizou na casa de 6.13 a partir da época 7 e parou de cair. Esse espaçamento entre as curvas indica a clara ocorrência de *overfitting* (memorização).
- **Análise Qualitativa:** Apesar do sobreajuste numérico, qualitativamente foi o melhor experimento. O modelo aprendeu a estrutura das palavras e pontuações, replicando frases exatas presentes no dataset de treino: *"Every effort moves you know, was not that my hostess was to the fact that..."*.

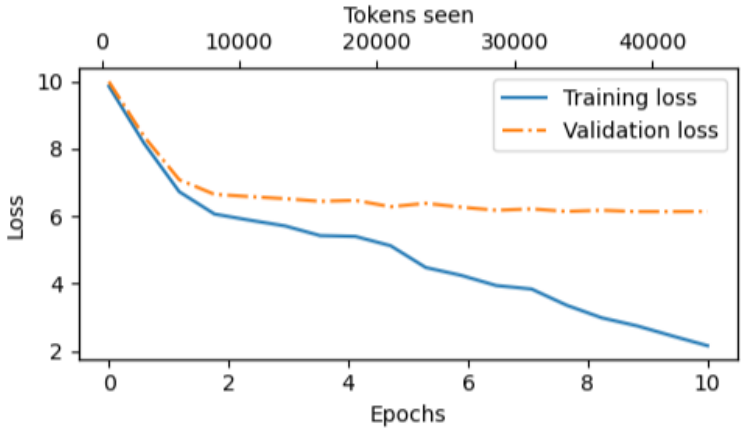


Figura 3 - Curvas de Loss com Alta Regularização.

4. Geração de texto

Nesta etapa, foram exploradas as diferenças práticas nas estratégias de geração de texto. A abordagem inicial do modelo consistia na decodificação gulosa (*Greedy Decoding*), que utiliza a função `argmax` para sempre selecionar o token com a maior pontuação matemática (probabilidade). O problema dessa abordagem é que a geração se torna previsível, resultando em textos repetitivos.

Para contornar isso e obter uma diversidade controlada, foram implementadas as estratégias de *Top-k Sampling* e *Temperature Scaling*.

4.1 Top-k Sampling

O *Top-k Sampling* restringe a amostragem aos k tokens mais prováveis, excluindo as demais opções ao mascarar suas probabilidades. No experimento realizado com $k = 3$, utiliza-se a função `torch.topk` para encontrar o limite mínimo entre os 3 maiores *logits*. Em seguida, aplica-se `torch.where` para substituir todos os valores abaixo desse limite por infinito negativo (`-inf`):

```
top_k = 3
top_logits, top_pos = torch.topk(next_token_logits, top_k)

new_logits = torch.where(
    condition=next_token_logits < top_logits[-1],
    input=torch.tensor(float('-inf')),
    other=next_token_logits
)
topk_probas = torch.softmax(new_logits, dim=0)
```

Ao aplicar a função *softmax* em `new_logits`, os valores `-inf` se transformam em probabilidade 0, garantindo que o modelo apenas selecione palavras coerentes dentre as opções restritas, eliminando tokens absurdos.

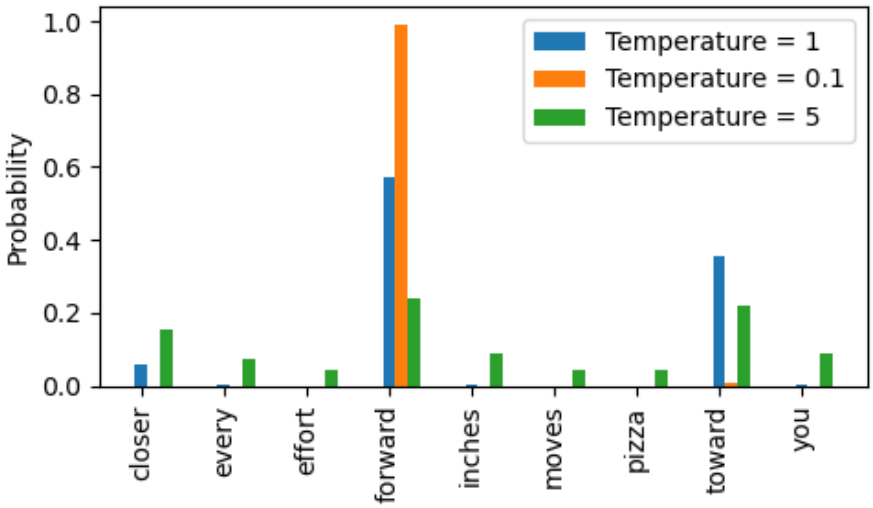


Figura 4 - Distribuição de probabilidades após aplicação do filtro Top-k ($k = 3$).

4.2 Temperature Scaling

Para permitir que o modelo não escolha sempre a mesma palavra entre os *Top-k*, trocam-se o `argmax` pela função `torch.multinomial`, que seleciona o próximo token baseando-se em um sorteio ponderado (amostragem probabilística).

Junto a isso, foi implementado o *Temperature Scaling*, que divide os *logits* por uma constante T (temperatura) antes do cálculo da *softmax*:

```
def softmax_with_temperature(logits, temperature):
    scaled_logits = logits / temperature
    return torch.softmax(scaled_logits, dim=0)
```

Foram testadas três temperaturas diferentes ($T = 1$, $T = 0.1$ e $T = 5$):

- **Temperatura = 1:** Mantém as probabilidades originais inalteradas.
- **Temperatura = 0.1 (Baixa):** Afia a distribuição. O token de maior probabilidade fica com quase 100% de chance de ser escolhido, tornando o modelo mais "confiante", porém menos "criativo"(comportamento parecido com o *Greedy Decoding*).
- **Temperatura = 5 (Alta):** Achata a distribuição, deixando quase uniforme. Aumenta a variedade e criatividade do texto gerado, mas eleva o risco de selecionar palavras sem contexto.

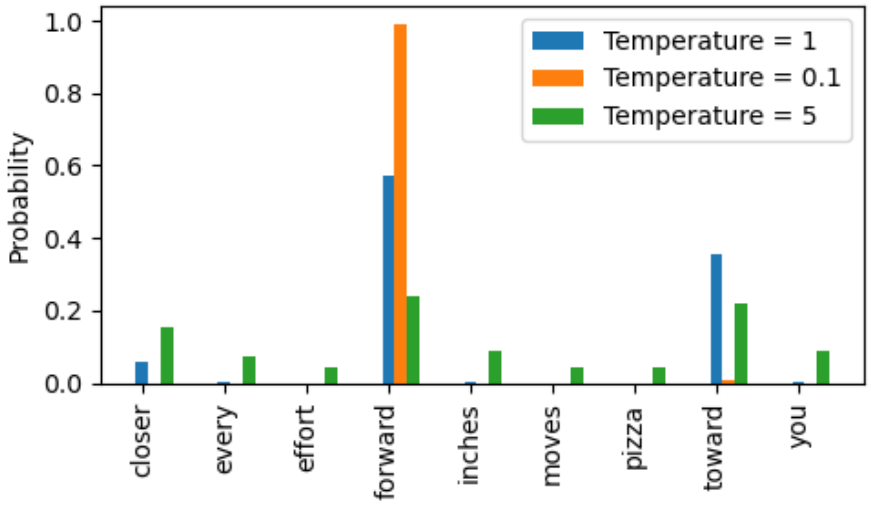


Figura 5 - Efeito do Temperature Scaling nas probabilidades dos tokens ($T = 1$, 0.1 e 5).

5. Carregamento de Pesos

Por fim, garantiu-se a reusabilidade do código através do salvamento do dicionário de estados (*state_dict*) do modelo e do otimizador PyTorch. O repositório foi adaptado com sucesso para carregar parâmetros pré-treinados do modelo GPT-2 (124M) da OpenAI. Isso permitiu que a arquitetura gerasse textos complexos com qualidade de produção, evidenciando que a implementação estrutural estava correta e que o carregamento superou a limitação do treinamento local.

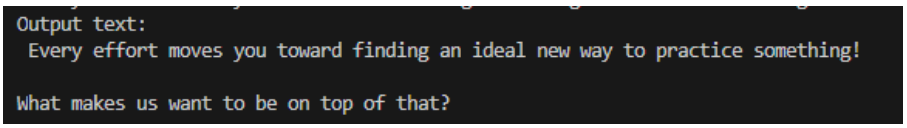


Figura 6 - Complemento da frase "Every effort moves you"realizado pelo modelo com os pesos GPT-2

6. Limitações e Conclusão

A execução da Sprint 5 cumpriu com rigor todos os requisitos de arquitetura e treinamento. No entanto, a principal limitação observada reside na natureza dos dados. Como o conjunto de dados utilizado (*the-verdict.txt*) é extremamente diminuto e as épocas foram prolongadas, o modelo inevitavelmente sobreajustou (*overfitting*) os dados, conforme evidenciado pela divergência gráfica entre a *loss* de validação e a de treinamento no Experimento 3.

Conclui-se que o modelo decorou passagens específicas do texto ao invés de aprender regras gerais de linguagem. Para aplicações reais onde o objetivo seja generalização sem memorização explícita, faria-se necessário um conjunto de treinamento na ordem de Gigabytes ou Terabytes, iterando-se por poucas épocas, mantendo o *pipeline* implementado nesse trabalho.

7. Referências Bibliográficas

RASCHKA, Sebastian. Build a large language model (from scratch). Simon and Schuster, 2024.